

Spis treści

- 1. Podstawy języka C#
 - 1.1. Czym jest C#
 - 1.2. Struktura programu
 - 1.3. Typy danych
 - 1.4. Zmienne i stałe
 - 1.5. Console.WriteLine()
 - 1.6. Console.ReadLine()
 - 1.7. Interpolacja stringów
 - 1.8. Konwersje typów
 - 1.9. Komentarze
 - 1.10. Obsługa wyjątków
- 2. Operatory
 - 2.1. Arytmetyczne
 - 2.2. Porównania
 - 2.3. Logiczne
 - 2.4. Przypisania
 - 2.5. Operatory warunkowe
 - 2.6. Null-coalescing
 - 2.7. Priorytety operatorów
- 3. Instrukcje sterujące
 - 3.1. if
 - 3.2. if else
 - 3.3. else if
 - 3.4. switch
 - 3.5. switch expression
 - 3.6. for
 - 3.7. while
 - 3.8. do while
 - 3.9. foreach
 - 3.10. break i continue
- 4. Tablice
 - 4.1. Tablice jednowymiarowe
 - 4.2. Tablice wielowymiarowe
 - 4.3. Tablice postrzępione
 - 4.4. Operacje na tablicach
 - 4.5. Klasa Array
- 5. Kolekcje
 - 5.1. List<T>
 - 5.2. Dictionary<TKey,TValue>
 - 5.3. HashSet<T>
 - 5.4. Queue<T>
 - 5.5. Stack<T>
- 6. Łańcuchy znaków
 - 6.1. String
 - 6.2. StringBuilder
 - 6.3. Najważniejsze metody

- 6.4. Split
- 6.5. Replace
- 6.6. Trim
- 6.7. ToUpper i ToLower
- 6.8. Przetwarzanie tekstu
- 7. Metody
 - 7.1. Tworzenie metod
 - 7.2. Parametry
 - 7.3. Zwracanie wartości
 - 7.4. ref i out
 - 7.5. params
 - 7.6. Przeciążanie metod
- 8. Operacje na plikach
 - 8.1. File
 - 8.2. FileInfo
 - 8.3. Directory
 - 8.4. StreamReader
 - 8.5. StreamWriter
 - 8.6. CSV
 - 8.7. JSON
- 9. Obsługa wyjątków
 - 9.1. try / catch / finally
 - 9.2. Własne wyjątki
- 10. Klasy i obiekty
 - 10.1. Definicja klasy
 - 10.2. Obiekt
 - 10.3. Pola
 - 10.4. Właściwości
 - 10.5. Konstruktory
- 11. Hermetyzacja
 - 11.1. Modyfikatory dostępu
 - 11.2. get i set
- 12. Dziedziczenie
 - 12.1. Klasy bazowe i pochodne
 - 12.2. override i virtual
 - 12.3. base
- 13. Polimorfizm
 - 13.1. Polimorfizm metod
 - 13.2. Przeciążanie
 - 13.3. Nadpisywanie
- 14. Klasy abstrakcyjne i interfejsy
 - 14.1. abstract
 - 14.2. interface
 - 14.3. Implementacja interfejsów
- 15. LINQ
 - 15.1. Where
 - 15.2. Select
 - 15.3. OrderBy i OrderByDescending
 - 15.4. First i FirstOrDefault
 - 15.5. Any i All

- 15.6. GroupBy
 - 16. Sortowanie
 - 16.1. Bubble Sort
 - 16.2. Selection Sort
 - 16.3. Insertion Sort
 - 16.4. Merge Sort
 - 16.5. Quick Sort
 - 17. Wyszukiwanie
 - 17.1. Liniowe
 - 17.2. Binarne
 - 18. Algorytmy matematyczne
 - 18.1. NWD
 - 18.2. NWW
 - 18.3. Liczby pierwsze
 - 18.4. Sito Eratostenesa
 - 18.5. Rozkład na czynniki pierwsze
 - 19. Algorytmy tekstowe
 - 19.1. Palindrom
 - 19.2. Anagram
 - 19.3. Zliczanie znaków
 - 19.4. Szyfr Cezara
 - 20. Tablice dwuwymiarowe
 - 20.1. Tworzenie
 - 20.2. Iteracja
 - 20.3. Suma przekątnych
 - 20.4. Obrót macierzy
 - 21. SQL
 - 21.1. SELECT
 - 21.2. WHERE
 - 21.3. ORDER BY
 - 21.4. GROUP BY
 - 21.5. INSERT
 - 21.6. UPDATE
 - 21.7. DELETE
 - 21.8. JOIN
 - 22. SQL Server + C#
 - 22.1. SqlConnection
 - 22.2. SqlCommand i SqlDataReader
 - 22.3. Parametryzowane zapytania
 - 23. Windows Forms
 - 23.1. Tworzenie formularza
 - 23.2. Kontrolki
 - 24. Obsługa zdarzeń
 - 24.1. Click
 - 24.2. TextChanged
 - 24.3. SelectedIndexChanged
 - 25. DataGridView
 - 25.1. Wyświetlanie danych
 - 25.2. Połączenie z bazą danych
-

Jak korzystać z tego poradnika: Każdy rozdział jest samodzielny — możesz czytać je w dowolnej kolejności. Rozdziały OOP (10–14) warto czytać po kolei, ponieważ każdy buduje na wiedzy z poprzedniego.

1. Podstawy języka C#

1.1. Czym jest C#

C# to nowoczesny, obiektowy język programowania stworzony przez Microsoft i wydany w 2000 roku jako część platformy .NET. Jest językiem **statycznie typowanym** — typy zmiennych muszą być deklarowane z góry (lub wywnioskowane przez kompilator). C# jest językiem **kompilowanym** do kodu pośredniego (CIL), który następnie wykonuje maszyna wirtualna CLR (Common Language Runtime).

C# jest stosowany powszechnie w tworzeniu aplikacji desktopowych (Windows Forms, WPF), webowych (ASP.NET), gier (Unity) oraz aplikacji bazodanowych — w tym w kwalifikacji INF.04.

Główne cechy C#:

CECHA	OPIS
Statyczne typowanie	Typ zmiennej określany w momencie deklaracji
Obiektowość	Wszystko jest klasą lub strukturą
Bezpieczeństwo typów	Kompilator wykrywa błędy typów przed uruchomieniem
Garbage Collector	Automatyczne zarządzanie pamięcią
Platforma .NET	Bogata biblioteka standardowa

1.2. Struktura programu

Każdy program w C# składa się z co najmniej jednej klasy i metody **Main**, która jest punktem wejścia do programu.

```
using System;           // importowanie przestrzeni nazw

namespace MojProgram    // przestrzeń nazw – grupuje klasy
{
    class Program        // definicja klasy
    {
        static void Main(string[] args) // punkt wejścia
        {
            Console.WriteLine("Witaj, świecie!");
        }
    }
}
```

W nowszych wersjach C# (9.0+) można używać tzw. **top-level statements** — uproszczonej formy bez klasy i metody **Main**:

```
using System;

Console.WriteLine("Witaj, świecie!"); // wystarczy tylko tyle!
```

1.3. Typy danych

C# jest językiem statycznie typowanym — każda zmienna musi mieć określony typ. Poniższa tabela przedstawia podstawowe typy wbudowane:

TYP	OPIS	ZAKRES / PRZYKŁAD
int	Liczba całkowita 32-bit	-2 147 483 648 do 2 147 483 647
long	Liczba całkowita 64-bit	Bardzo duże liczby całkowite
double	Liczba zmiennoprzecinkowa 64-bit	3.14, -0.5, 2.0
float	Liczba zmiennoprzecinkowa 32-bit	3.14f (wymaga przyrostka f)
decimal	Liczba dziesiętna wysokiej precyzji	Stosowana w finansach
bool	Wartość logiczna	true, false
char	Pojedynczy znak	'A', 'z', '5'
string	Łańcuch znaków	"Witaj", ""
object	Typ bazowy wszystkich typów	Może przechowywać cokolwiek

```
int wiek = 25;
double temperatura = 36.6;
float pi = 3.14f;
decimal cena = 19.99m;           // przyrostek m dla decimal
bool jestAktywny = true;
char litera = 'A';
string imie = "Jan";
```

Typy wartościowe vs referencyjne:

KATEGORIA	TYPY	PRZECHOWYWANE
Wartościowe	int, double, bool, char, float, decimal, struct	Bezpośrednio na stosie
Referencyjne	string, class, array, List<T>	Referencja na stosie, dane na stercie

1.4. Zmienne i stałe

Zmienne deklaruje się podając typ i nazwę. Można też użyć słowa kluczowego `var` — kompilator sam wywnioskuje typ:

```
// Deklaracja z jawnym typem
int liczba = 10;
string tekst = "Witaj";

// Deklaracja z var - typ wnioskowany przez kompilator
var wynik = 42;           // int
var nazwa = "Kowalski";   // string
var lista = new List<int>();

// Deklaracja bez inicjalizacji (tylko dla pól klasy)
int x;
x = 5;
```

Stałe (`const`) — wartości, których nie można zmienić po zadeklarowaniu:

```
const double PI = 3.14159265;
const int MAX_ROZMIAR = 100;
const string WERSJA = "1.0.0";

// PI = 3.0; // błąd kompilacji – stałej nie można zmienić
```

Readonly — pola, które można ustawić tylko w konstruktorze:

```
class Konfiguracja
{
    readonly string nazwaAplikacji;

    public Konfiguracja(string nazwa)
    {
        nazwaAplikacji = nazwa;    // tylko w konstruktorze
    }
}
```

1.5. Console.WriteLine()

`Console.WriteLine()` to podstawowa metoda wyświetlania danych w konsoli. Dodaje znak nowej linii na końcu. `Console.Write()` działa tak samo, ale nie dodaje nowej linii.

```
// Podstawowe wyświetlanie
Console.WriteLine("Witaj!");           // z nową linią
Console.Write("Bez nowej linii ");     // bez nowej linii
Console.Write("kontynuacja\n");        // \n = ręczna nowa linia

// Wyświetlanie zmiennych
int wiek = 25;
string imie = "Jan";
Console.WriteLine(wiek);               // 25
Console.WriteLine(imie);               // Jan

// Formatowanie – metoda String.Format
Console.WriteLine("Imię: {0}, Wiek: {1}", imie, wiek);
// Wynik: Imię: Jan, Wiek: 25

// Formatowanie liczb
double srednia = 4.5678;
Console.WriteLine("{0:F2}", srednia);  // 4.57 (2 miejsca po przecinku)
Console.WriteLine("{0:D5}", 42);       // 00042 (5 cyfr z zerami)
Console.WriteLine("{0:C}", 19.99);     // 19,99 zł (waluta – zależna od ustawień systemu)
```

Specyfikatory formatu:

SPECYFIKATOR	OPIS	PRZYKŁAD	WYNIK
{0:F2}	2 miejsca po przecinku	"{0:F2}", 3.14159	3,14
{0:F0}	0 miejsc (zaokrąglenie)	"{0:F0}", 3.7	4
{0:D5}	5 cyfr z zerami	"{0:D5}", 42	00042
{0:C}	Waluta	"{0:C}", 9.99	9,99 zł
{0:P}	Procent	"{0:P}", 0.75	75,00%
{0:N}	Liczba z separatorami	"{0:N}", 1234567	1 234 567,00

1.6. Console.ReadLine()

`Console.ReadLine()` wczytuje jeden wiersz tekstu wprowadzonego przez użytkownika. Zawsze zwraca wartość typu `string`.

```
// Wczytanie tekstu
Console.Write("Podaj imię: ");
string imie = Console.ReadLine();
Console.WriteLine($"Cześć, {imie}!");

// Wczytanie liczby – konieczna konwersja
Console.Write("Podaj wiek: ");
string wejscie = Console.ReadLine();
int wiek = int.Parse(wejscie);    // konwersja string → int

// Bezpieczna konwersja z TryParse
Console.Write("Podaj liczbę: ");
string tekst = Console.ReadLine();
if (int.TryParse(tekst, out int liczba))
{
    Console.WriteLine($"Podałeś: {liczba}");
}
else
{
    Console.WriteLine("To nie jest liczba!");
}
```

`Console.ReadKey()` — wczytuje pojedynczy klawisz (bez Enter):

```
Console.Write("Naciśnij dowolny klawisz...");
ConsoleKeyInfo klawisz = Console.ReadKey();
Console.WriteLine($"Naciśnąłeś: {klawisz.Key}");
```

1.7. Interpolacja stringów

Interpolacja stringów (f-stringi w C#) to najwygodniejszy sposób łączenia tekstu ze zmiennymi. Tworzymy je, dodając `$` przed cudzysłowem, a wyrażenia umieszczamy w klamrach `{ }`.

```
string imie = "Jan";
int wiek = 25;
double srednia = 87.456;
```

```
// Podstawowa interpolacja
Console.WriteLine($"Cześć, {imie}!");           // Cześć, Jan!
Console.WriteLine($"Mam {wiek} lat");           // Mam 25 lat

// Wyrażenia wewnątrz klamr
Console.WriteLine($"Za 5 lat: {wiek + 5} lat");   // Za 5 lat: 30 lat
Console.WriteLine($"Imię wielkie: {imie.ToUpper()}"); // Imię wielkie: JAN

// Formatowanie liczb
Console.WriteLine($"Średnia: {srednia:F2}");     // Średnia: 87,46
Console.WriteLine($"Liczba: {42:D5}");          // Liczba: 00042
Console.WriteLine($"Proc: {0.75:P0}");          // Proc: 75%

// Wyrównanie kolumn
Console.WriteLine($"{"Imię",-15}{"Wiek",5}");
Console.WriteLine($"{{imie,-15}}{{wiek,5}}");

// Imię           25
// Jan           25
```

Porównanie metod łączenia stringów:

METODA	PRZYKŁAD	UWAGI
Konkatenacja <code>+</code>	<code>"Cześć " + imie</code>	Tworzy wiele tymczasowych stringów
<code>String.Format</code>	<code>String.Format("Cześć {0}", imie)</code>	Starszy styl
Interpolacja <code>\$</code>	<code>\$"Cześć {imie}"</code>	Zalecany — czytelny i wydajny
<code>StringBuilder</code>	<code>sb.Append("Cześć ").Append(imie)</code>	Najwydajniejszy przy dużej liczbie operacji

1.8. Konwersje typów

C# oferuje kilka sposobów konwersji typów. Wyróżniamy konwersje **niejawne** (automatyczne, gdy nie ma ryzyka utraty danych) i **jawne** (wymagają rzutowania, bo może nastąpić utrata danych).

Konwersja niejawna (bezpieczna):

```
int calkowita = 42;
long duza = calkowita;    // int → long (bezpieczne)
double zmienne = calkowita; // int → double (bezpieczne)
float f = calkowita;      // int → float (bezpieczne)
```

Konwersja jawna — rzutowanie `(typ)`:

```
double d = 3.99;
int i = (int)d;    // UWAGA: obcina, nie zaokrągla! → 3

long duza = 10000000000L;
int mala = (int)duza;    // ryzyko utraty danych przy dużych wartościach
```

Konwersja ze stringów — `Parse()` i `TryParse()`:


```
// Parse – rzuca wyjątek gdy się nie uda
int a = int.Parse("42");
double b = double.Parse("3.14");
bool c = bool.Parse("true");

// TryParse – bezpieczna, zwraca true/false
if (int.TryParse("123abc", out int wynik))
    Console.WriteLine(wynik);    // nie wywoła się
else
    Console.WriteLine("Błąd konwersji");

// Convert.ToXxx – alternatywa dla Parse
int x = Convert.ToInt32("42");
double y = Convert.ToDouble("3.14");
string s = Convert.ToString(42);
```

Konwersja na string:

```
int liczba = 42;
string s1 = liczba.ToString();    // "42"
string s2 = liczba.ToString("D5"); // "00042"
string s3 = Convert.ToString(liczba);
```

Pełna tabela konwersji:

Z → DO	METODA	PRZYKŁAD
string → int	int.Parse() / int.TryParse()	int.Parse("42")
string → double	double.Parse()	double.Parse("3.14")
int → string	.ToString()	(42).ToString()
double → int	Rzutowanie (int)	(int)3.7 → 3
int → double	Niejawna	double d = 5;
Dowolny → string	Convert.ToString()	Convert.ToString(true)

1.9. Komentarze

```
// Komentarz jednoliniowy – od // do końca linii

/* Komentarz
   wieloliniowy */

/// <summary>
/// Komentarz dokumentacyjny XML – pojawia się w IntelliSense
/// </summary>
/// <param name="x">Opis parametru x</param>
/// <returns>Opis wartości zwracanej</returns>
static int Kwadrat(int x)
{
```

```
    return x * x;
}
```

1.10. Obsługa wyjątków

Szczegółowy opis w rozdziale 9. Podstawowy schemat:

```
try
{
    int wynik = int.Parse(Console.ReadLine());
    Console.WriteLine($"Podałeś: {wynik}");
}
catch (FormatException)
{
    Console.WriteLine("To nie jest liczba!");
}
catch (Exception ex)
{
    Console.WriteLine($"Błąd: {ex.Message}");
}
finally
{
    Console.WriteLine("Koniec programu");
}
```

2. Operatory

2.1. Arytmetyczne

OPERATOR	OPIS	PRZYKŁAD	WYNIK
+	Dodawanie	5 + 3	8
-	Odejmowanie	5 - 3	2
*	Mnożenie	5 * 3	15
/	Dzielenie	7 / 2	3 (całkowite!)
%	Reszta z dzielenia	7 % 2	1
++	Inkrementacja	x++ lub ++x	x + 1
--	Dekrementacja	x-- lub --x	x - 1

```
int a = 7, b = 2;
Console.WriteLine(a / b);    // 3 - dzielenie całkowite!
Console.WriteLine(a % b);    // 1 - reszta

double c = 7.0, d = 2.0;
Console.WriteLine(c / d);    // 3.5 - dzielenie zmiennoprzecinkowe

// Inkrementacja - różnica między pre- a post-
```

```
int x = 5;

Console.WriteLine(x++); // 5 – wyświetla PRZED inkrementacją
Console.WriteLine(x);   // 6

int y = 5;

Console.WriteLine(++y); // 6 – wyświetla PO inkrementacji
```

2.2. Porównania

Zawsze zwracają wartość `bool` (`true` lub `false`).

OPERATOR	OPIS	PRZYKŁAD	WYNIK
<code>=</code>	Równy	<code>5 = 5</code>	<code>true</code>
<code>≠</code>	Różny	<code>5 ≠ 3</code>	<code>true</code>
<code>></code>	Większy	<code>5 > 3</code>	<code>true</code>
<code><</code>	Mniejszy	<code>5 < 3</code>	<code>false</code>
<code>≥</code>	Większy lub równy	<code>5 ≥ 5</code>	<code>true</code>
<code>≤</code>	Mniejszy lub równy	<code>3 ≤ 5</code>	<code>true</code>

```
int wiek = 18;

bool jestPełnoletni = wiek ≥ 18; // true
bool rowny = wiek == 18;        // true

// UWAGA: porównanie stringów
string s1 = "Ala";
string s2 = "Ala";
bool rowne = s1 == s2;           // true – C# porównuje zawartość
bool rowneIgnoreCase = s1.Equals(s2, StringComparison.OrdinalIgnoreCase);
```

2.3. Logiczne

OPERATOR	OPIS	PRZYKŁAD
<code>&&</code>	AND (i) — oba muszą być true	<code>a > 0 && b > 0</code>
<code> </code>	OR (lub) — przynajmniej jeden true	<code>a > 0 b > 0</code>
<code>!</code>	NOT (negacja)	<code>!jestAktywny</code>

```
int wiek = 20;
bool maPrawoJazdy = true;

bool mozeProwadzic = wiek ≥ 18 && maPrawoJazdy; // true
bool wiek_OK = wiek < 16 || wiek ≥ 65;          // false
bool nieAktywny = !maPrawoJazdy;                // false

// Krótkie spięcie (short-circuit evaluation)
// && – jeśli lewy operand to false, prawy nie jest sprawdzany
// || – jeśli lewy operand to true, prawy nie jest sprawdzany

string s = null;
```

```
if (s != null && s.Length > 0)    // bezpieczne – s.Length nie wywoła się gdy s == null
    Console.WriteLine(s);
```

2.4. Przypisania

OPERATOR	OPIS	ODPOWIEDNIK
=	Przypisanie	x = 5
+=	Dodaj i przypisz	x = x + 5
-=	Odejmij i przypisz	x = x - 5
*=	Pomnóż i przypisz	x = x * 5
/=	Podziel i przypisz	x = x / 5
%=	Reszta i przypisz	x = x % 5

```
int x = 10;
x += 5;    // x = 15
x -= 3;    // x = 12
x *= 2;    // x = 24
x /= 4;    // x = 6
x %= 4;    // x = 2
```

2.5. Operatory warunkowe

Operator trójargumentowy `? :` — skrócona forma `if-else` :

```
int wiek = 20;
string status = wiek >= 18 ? "pełnoletni" : "niepełnoletni";
Console.WriteLine(status);    // pełnoletni

int max = a > b ? a : b;    // większa z dwóch liczb
```

2.6. Null-coalescing

Operatory do obsługi wartości `null` :

```
// ?? – zwraca lewą stronę jeśli nie jest null, w przeciwnym razie prawą
string imie = null;
string wyswietlane = imie ?? "Nieznany";    // "Nieznany"

string tekst = "Jan";
string wynik = tekst ?? "Brak";    // "Jan"

// ??= – przypisuje wartość tylko gdy zmienna jest null
string s = null;
s ??= "domyślna";    // s = "domyślna"

// ?. – operator warunkowy dla właściwości (null-safe)
```

```
string str = null;
int? dlugosc = str?.Length;    // null (nie rzuca NullReferenceException)
```

2.7. Priorytety operatorów

Operatory wykonywane są według ustalonej kolejności — od najwyższego priorytetu do najniższego:

PRIORYTET	OPERATORY
1 (najwyższy)	<code>()</code> nawiasy, <code>.</code> dostęp do składowej, <code>[]</code> indeks
2	<code>++</code> , <code>--</code> (postfiksowe), <code>!</code> , <code>~</code> (prefiksowe)
3	<code>*</code> , <code>/</code> , <code>%</code>
4	<code>+</code> , <code>-</code>
5	<code><</code> , <code>></code> , <code>≤</code> , <code>≥</code>
6	<code>=</code> , <code>≠</code>
7	<code>&&</code>
8	<code> </code>
9	<code>??</code>
10 (najniższy)	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code>

```
int wynik = 2 + 3 * 4;    // 14 (nie 20) – mnożenie ma wyższy priorytet
int wynik2 = (2 + 3) * 4; // 20 – nawiasy zmieniają kolejność
bool b = 2 + 3 > 4;       // true – najpierw 2+3=5, potem 5>4
```

3. Instrukcje sterujące

3.1. if

```
int wiek = 20;

if (wiek ≥ 18)
{
    Console.WriteLine("Pełnoletni");
}
```

Nawiasy klamrowe `{ }` można pominąć dla jednoliniowych instrukcji (niezalecane — mniej czytelne):

```
if (wiek ≥ 18)
    Console.WriteLine("Pełnoletni");
```

3.2. if else

```
int wiek = 16;

if (wiek ≥ 18)
{
    Console.WriteLine("Pełnoletni – możesz wejść");
}
else
{
    Console.WriteLine("Niepełnoletni – wstęp wzbroniony");
}
```

3.3. else if

```
int ocena = 85;

if (ocena ≥ 90)
{
    Console.WriteLine("Ocena: celujący");
}
else if (ocena ≥ 80)
{
    Console.WriteLine("Ocena: bardzo dobry");
}
else if (ocena ≥ 70)
{
    Console.WriteLine("Ocena: dobry");
}
else if (ocena ≥ 60)
{
    Console.WriteLine("Ocena: dostateczny");
}
else
{
    Console.WriteLine("Ocena: niedostateczny");
}
```

3.4. switch

`switch` sprawdza wartość zmiennej i wykonuje odpowiedni blok `case`. Każdy `case` powinien kończyć się `break`:

```
int dzien = 3;

switch (dzien)
{
    case 1:
        Console.WriteLine("Poniedziałek");
        break;
    case 2:
        Console.WriteLine("Wtorek");
        break;
```

```

case 3:
    Console.WriteLine("Środa");
    break;
case 4:
    Console.WriteLine("Czwartek");
    break;
case 5:
    Console.WriteLine("Piątek");
    break;
case 6:
case 7:
    Console.WriteLine("Weekend");    // dwa case dla jednego bloku
    break;
default:
    Console.WriteLine("Nieznany dzień");
    break;
}

```

switch działa też na stringach i innych typach:

```

string kolor = "czerwony";

switch (kolor)
{
    case "czerwony":
        Console.WriteLine("STOP");
        break;
    case "żółty":
        Console.WriteLine("UWAGA");
        break;
    case "zielony":
        Console.WriteLine("JEDŹ");
        break;
    default:
        Console.WriteLine("Nieznany kolor");
        break;
}

```

3.5. switch expression

Nowszy (C# 8.0+) zwięzły zapis switch zwracający wartość:

```

int dzien = 3;
string nazwaDnia = dzien switch
{
    1 => "Poniedziałek",
    2 => "Wtorek",
    3 => "Środa",
    4 => "Czwartek",
    5 => "Piątek",
    6 or 7 => "Weekend",
    _ => "Nieznany"    // _ to odpowiednik default
}

```

```
};  
Console.WriteLine(nazwaDnia); // Środa
```

3.6. for

Pętla **for** używana, gdy znamy z góry liczbę iteracji:

```
// Schemat: for (inicjalizacja; warunek; krok)  
for (int i = 0; i < 5; i++)  
{  
    Console.WriteLine(i); // 0, 1, 2, 3, 4  
}  
  
// Pętla malejąca  
for (int i = 10; i ≥ 1; i--)  
{  
    Console.Write(i + " "); // 10 9 8 7 6 5 4 3 2 1  
}  
  
// Iteracja po tablicy z indeksem  
int[] liczby = { 10, 20, 30, 40, 50 };  
for (int i = 0; i < liczby.Length; i++)  
{  
    Console.WriteLine($"liczby[{i}] = {liczby[i]}");  
}  
  
// Pętla z krokiem innym niż 1  
for (int i = 0; i ≤ 100; i += 10)  
{  
    Console.Write(i + " "); // 0 10 20 30 40 50 60 70 80 90 100  
}
```

3.7. while

Pętla **while** wykonuje blok kodu **dopóki** warunek jest spełniony. Warunek sprawdzany jest **przed** każdą iteracją:

```
int i = 0;  
while (i < 5)  
{  
    Console.WriteLine(i);  
    i++;  
}  
  
// Wzorzec: wczytywanie danych aż do poprawnego wejścia  
int liczba;  
while (!int.TryParse(Console.ReadLine(), out liczba))  
{  
    Console.WriteLine("To nie jest liczba! Spróbuj ponownie:");  
}  
Console.WriteLine($"Wpisałeś: {liczba}");
```


3.8. do while

Podobna do `while`, ale warunek sprawdzany jest **po** każdej iteracji — ciało pętli wykona się zawsze **przynajmniej raz**:

```
int i = 0;

do
{
    Console.WriteLine(i);
    i++;
} while (i < 5);

// Użyteczny wzorzec – menu z powtórzeniem
string odpowiedz;

do
{
    Console.WriteLine("1. Opcja A");
    Console.WriteLine("2. Opcja B");
    Console.WriteLine("0. Wyjście");
    Console.Write("Wybór: ");
    odpowiedz = Console.ReadLine();
    // ... obsługa wyboru
} while (odpowiedz != "0");
```

3.9. foreach

`foreach` iteruje po każdym elemencie kolekcji (tablicy, listy itp.). Nie daje dostępu do indeksu:

```
int[] liczby = { 1, 2, 3, 4, 5 };
foreach (int l in liczby)
{
    Console.WriteLine(l);
}

// Dla List<T>
List<string> imiona = new List<string> { "Anna", "Jan", "Maria" };
foreach (string imie in imiona)
{
    Console.WriteLine(imie.ToUpper());
}

// Dla Dictionary
Dictionary<string, int> oceny = new Dictionary<string, int>
{
    { "Jan", 5 }, { "Anna", 4 }
};
foreach (KeyValuePair<string, int> para in oceny)
{
    Console.WriteLine($"{para.Key}: {para.Value}");
}
```

3.10. break i continue

`break` — natychmiastowe wyjście z pętli:

```
for (int i = 0; i < 10; i++)
{
    if (i == 5) break;    // wyjście gdy i == 5
    Console.Write(i + " "); // 0 1 2 3 4
}
```

continue — pominięcie bieżącej iteracji, przejście do następnej:

```
for (int i = 0; i < 10; i++)
{
    if (i % 2 == 0) continue; // pominięcie liczby parzyste
    Console.Write(i + " ");   // 1 3 5 7 9
}
```

4. Tablice

4.1. Tablice jednowymiarowe

Tablica to kolekcja elementów **tego samego typu** o **stałym rozmiarze**:

```
// Deklaracja i inicjalizacja
int[] liczby = new int[5];           // 5 elementów, domyślnie 0
int[] wartosci = new int[] { 1, 2, 3, 4, 5 };
int[] skrocona = { 10, 20, 30 };    // skrócona forma
string[] imiona = { "Jan", "Anna", "Piotr" };

// Dostęp do elementów (indeks od 0)
Console.WriteLine(liczby[0]);       // pierwszy element
Console.WriteLine(imiona[2]);       // trzeci element: "Piotr"

// Modyfikacja elementu
liczby[0] = 42;

// Rozmiar tablicy
Console.WriteLine(skrocona.Length); // 3

// Iteracja
for (int i = 0; i < imiona.Length; i++)
{
    Console.WriteLine($"{i}: {imiona[i]}");
}
```

4.2. Tablice wielowymiarowe

```
// Tablica 2D – prostokątna (rectangular array)
int[,] macierz = new int[3, 4];    // 3 wiersze, 4 kolumny
int[,] init = { { 1, 2 }, { 3, 4 }, { 5, 6 } };
```

```
// Dostęp
macierz[0, 0] = 10;
Console.WriteLine(init[1, 1]);           // 4

// Wymiary
Console.WriteLine(init.GetLength(0)); // 3 - liczba wierszy
Console.WriteLine(init.GetLength(1)); // 2 - liczba kolumn

// Iteracja
for (int i = 0; i < init.GetLength(0); i++)
{
    for (int j = 0; j < init.GetLength(1); j++)
    {
        Console.Write($"{init[i, j],4}");
    }
    Console.WriteLine();
}
```

4.3. Tablice postrzępione

Tablice postrzępione (jagged arrays) to tablice tablic — każdy wiersz może mieć inną długość:

```
// Deklaracja
int[][] postrzepiona = new int[3][];
postrzepiona[0] = new int[] { 1, 2 };
postrzepiona[1] = new int[] { 3, 4, 5, 6 };
postrzepiona[2] = new int[] { 7 };

// Iteracja
for (int i = 0; i < postrzepiona.Length; i++)
{
    for (int j = 0; j < postrzepiona[i].Length; j++)
    {
        Console.Write(postrzepiona[i][j] + " ");
    }
    Console.WriteLine();
}
```

4.4. Operacje na tablicach

```
int[] tab = { 5, 3, 1, 4, 2 };

// Sortowanie rosnące
Array.Sort(tab);           // { 1, 2, 3, 4, 5 }

// Odwrócenie tablicy
Array.Reverse(tab);        // { 5, 4, 3, 2, 1 }

// Wyszukiwanie
int indeks = Array.IndexOf(tab, 3); // indeks elementu 3

// Kopiowanie
```

```

int[] kopia = new int[tam.Length];
Array.Copy(tam, kopia, tam.Length);

// Wypełnianie
int[] zera = new int[5];
Array.Fill(zera, 99);    // { 99, 99, 99, 99, 99 }

// Suma, min, max z LINQ
using System.Linq;
int suma = tam.Sum();
int min = tam.Min();
int max = tam.Max();

```

4.5. Klasa Array

METODA	OPIS
<code>Array.Sort(tam)</code>	Sortuje tablicę
<code>Array.Reverse(tam)</code>	Odwraca kolejność
<code>Array.IndexOf(tam, val)</code>	Szuka indeksu wartości
<code>Array.Copy(src, dst, len)</code>	Kopiuje elementy
<code>Array.Fill(tam, val)</code>	Wypełnia wartością
<code>Array.Clear(tam, idx, len)</code>	Zeruje elementy
<code>Array.Exists(tam, predicate)</code>	Sprawdza czy element istnieje
<code>Array.Find(tam, predicate)</code>	Zwraca pierwszy pasujący element

5. Kolekcje

Kolekcje są elastyczniejsze niż tablice — mogą dynamicznie zmieniać rozmiar. Wymagają `using System.Collections.Generic;`.

5.1. List<T>

`List<T>` to dynamiczna lista — odpowiednik tablicy, która może rosnąć i maleć:

```

// Tworzenie
List<int> liczby = new List<int>();
List<string> imiona = new List<string> { "Jan", "Anna" };

// Dodawanie
liczby.Add(10);
liczby.Add(20);
liczby.Add(30);
imiona.AddRange(new[] { "Piotr", "Maria" }); // dodaj wiele

// Usuwanie
liczby.Remove(20);    // usuwa pierwszą wartość 20
liczby.RemoveAt(0);   // usuwa element o indeksie 0

// Dostęp i modyfikacja

```

```

Console.WriteLine(imiona[0]);    // Jan
imiona[0] = "Adam";

// Przydatne właściwości i metody
Console.WriteLine(liczby.Count);    // liczba elementów
Console.WriteLine(liczby.Contains(30)); // true
int idx = imiona.IndexOf("Anna");    // indeks elementu
imiona.Sort();                      // sortowanie
imiona.Reverse();                   // odwrócenie
imiona.Clear();                     // wyczyszczenie

// Konwersja na tablicę
int[] tablica = liczby.ToArray();

```

5.2. Dictionary<TKey,TValue>

Dictionary przechowuje pary klucz-wartość. Klucze muszą być unikalne:

```

// Tworzenie
Dictionary<string, int> oceny = new Dictionary<string, int>();

// Dodawanie
oceny["Jan"] = 5;
oceny["Anna"] = 4;
oceny.Add("Piotr", 3);    // alternatywnie

// Dostęp – rzuca KeyNotFoundException gdy brak klucza
int ocenaJana = oceny["Jan"];

// Bezpieczny dostęp – TryGetValue
if (oceny.TryGetValue("Jan", out int ocena))
    Console.WriteLine($"Ocena Jana: {ocena}");

// Sprawdzanie klucza
if (oceny.ContainsKey("Anna"))
    Console.WriteLine("Anna istnieje");

// Iteracja
foreach (KeyValuePair<string, int> para in oceny)
    Console.WriteLine($"{para.Key}: {para.Value}");

// Klucze i wartości osobno
foreach (string klucz in oceny.Keys)
    Console.WriteLine(klucz);

foreach (int wartosc in oceny.Values)
    Console.WriteLine(wartosc);

// Usuwanie
oceny.Remove("Piotr");

Console.WriteLine(oceny.Count);    // liczba par

```

5.3. HashSet<T>

HashSet<T> to zbiór unikalnych elementów. Gwarantuje brak duplikatów i szybkie sprawdzanie przynależności:

```
HashSet<int> zbior = new HashSet<int>();
zbior.Add(1);
zbior.Add(2);
zbior.Add(2);    // duplikat – nie zostanie dodany
zbior.Add(3);

Console.WriteLine(zbior.Count);    // 3
Console.WriteLine(zbior.Contains(2)); // true

// Operacje na zbiorach
HashSet<int> a = new HashSet<int> { 1, 2, 3, 4 };
HashSet<int> b = new HashSet<int> { 3, 4, 5, 6 };

a.UnionWith(b);    // suma – { 1, 2, 3, 4, 5, 6 }
a.IntersectWith(b); // część wspólna
a.ExceptWith(b);   // różnica (a minus b)
```

5.4. Queue<T>

Queue<T> to kolejka FIFO (First In, First Out) — pierwszy dodany, pierwszy wychodzi:

```
Queue<string> kolejka = new Queue<string>();
kolejka.Enqueue("Pierwszy");
kolejka.Enqueue("Drugi");
kolejka.Enqueue("Trzeci");

Console.WriteLine(kolejka.Count);    // 3
Console.WriteLine(kolejka.Peek());   // "Pierwszy" – podgląd bez usuwania
Console.WriteLine(kolejka.Dequeue()); // "Pierwszy" – pobiera i usuwa
Console.WriteLine(kolejka.Dequeue()); // "Drugi"
```

5.5. Stack<T>

Stack<T> to stos LIFO (Last In, First Out) — ostatni dodany, pierwszy wychodzi:

```
Stack<int> stos = new Stack<int>();
stos.Push(1);
stos.Push(2);
stos.Push(3);

Console.WriteLine(stos.Count);    // 3
Console.WriteLine(stos.Peek());   // 3 – podgląd bez usuwania
Console.WriteLine(stos.Pop());    // 3 – pobiera i usuwa
Console.WriteLine(stos.Pop());    // 2
```

6. Łańcuchy znaków

6.1. String

`string` w C# jest typem **niezmiennym** (immutable) — każda operacja na stringu tworzy nowy obiekt:

```
string s = "Witaj, świecie!";

// Właściwości
Console.WriteLine(s.Length);      // 16 – liczba znaków

// Dostęp do znaków (jak tablica)
Console.WriteLine(s[0]);           // 'W'
Console.WriteLine(s[7]);           // 'ś'

// Porównywanie
string a = "abc", b = "ABC";
Console.WriteLine(a == b);         // false
Console.WriteLine(a.Equals(b, StringComparison.OrdinalIgnoreCase)); // true

// Sprawdzanie zawartości
Console.WriteLine(s.Contains("świecie")); // true
Console.WriteLine(s.StartsWith("Witaj")); // true
Console.WriteLine(s.EndsWith("!"));      // true

// Pusta lub null
Console.WriteLine(string.IsNullOrEmpty("")); // true
Console.WriteLine(string.IsNullOrWhiteSpace("  ")); // true
```

6.2. StringBuilder

`StringBuilder` jest mutowalny — idealny gdy wykonujesz wiele operacji łączenia:

```
using System.Text;

StringBuilder sb = new StringBuilder();

sb.Append("Witaj");
sb.Append(", ");
sb.Append("świecie");
sb.Append("!");
sb.AppendLine();           // + nowa linia
sb.Insert(0, ">> ");       // wstaw na pozycji 0

string wynik = sb.ToString();
Console.WriteLine(wynik);  // >> Witaj, świecie!

// Przydatne metody
sb.Replace("świecie", "C#");
sb.Clear();
Console.WriteLine(sb.Length);
```

Kiedy używać `StringBuilder`? Gdy łączysz wiele stringów w pętli — jest wielokrotnie szybszy od operatora `+`.

6.3. Najważniejsze metody

METODA	OPIS	PRZYKŁAD	WYNIK
Length	Długość stringa	"Ala".Length	3
ToUpper()	Wielkie litery	"aLa".ToUpper()	"ALA"
ToLower()	Małe litery	"ALA".ToLower()	"ala"
Trim()	Usuwa białe znaki z obu stron	" ala ".Trim()	"ala"
TrimStart()	Usuwa z lewej	" ala".TrimStart()	"ala"
TrimEnd()	Usuwa z prawej	"ala ".TrimEnd()	"ala"
Replace(old, new)	Zamiana	"abc".Replace("b", "X")	"aXc"
Contains(s)	Czy zawiera	"abcd".Contains("bc")	true
StartsWith(s)	Czy zaczyna się	"abc".StartsWith("ab")	true
EndsWith(s)	Czy kończy się	"abc".EndsWith("bc")	true
IndexOf(s)	Indeks pierwszego wystąpienia	"abc".IndexOf("b")	1
Substring(idx)	Podciąg od indeksu	"abcde".Substring(2)	"cde"
Substring(idx, len)	Podciąg o podanej długości	"abcde".Substring(1, 3)	"bcd"
Split(sep)	Dzielenie	"a,b,c".Split(',')	["a","b","c"]
string.Join(sep, arr)	Łączenie	string.Join("-", new[]{"a","b"})	"a-b"
PadLeft(n)	Uzupełnienie spacjami z lewej	"42".PadLeft(5)	" 42"
PadRight(n)	Uzupełnienie spacjami z prawej	"42".PadRight(5)	"42 "

6.4. Split

```
string dane = "Jan;Kowalski;25;Warszawa";
string[] pola = dane.Split(';');
// pola[0] = "Jan", pola[1] = "Kowalski", itd.

// Kilka separatorów naraz
string tekst = "jeden dwa,trzy,cztery";
string[] slowa = tekst.Split(new char[] { ' ', ',', ';'});

// Z usuwaniem pustych wpisów
string[] clean = tekst.Split(new char[] { ' ' },
    StringSplitOptions.RemoveEmptyEntries);
```

6.5. Replace

```
string zdanie = "Ala ma kota. Ala lubi kota.";
string zmienione = zdanie.Replace("kota", "psa");
// "Ala ma psa. Ala lubi psa."

// Replace można łączyć
string wynik = " hello world "
```



```
.Trim()
.Replace("hello", "witaj")
.ToUpper();
// "WITAJ WORLD"
```

6.6. Trim

```
string s = "  Witaj, świecie!  ";
Console.WriteLine(s.Trim());           // "Witaj, świecie!"
Console.WriteLine(s.TrimStart());      // "Witaj, świecie!  "
Console.WriteLine(s.TrimEnd());        // "  Witaj, świecie!"

// Trim konkretnych znaków
string s2 = "***tekst***";
Console.WriteLine(s2.Trim('*'));       // "tekst"
```

6.7. ToUpper i ToLower

```
string imie = "Jan Kowalski";
Console.WriteLine(imie.ToUpper());    // JAN KOWALSKI
Console.WriteLine(imie.ToLower());    // jan kowalski

// Porównywanie bez uwzględniania wielkości liter
string input = "TAK";
if (input.ToLower() == "tak")
    Console.WriteLine("Potwierdzono");
```

6.8. Przetwarzanie tekstu

```
// Liczenie wystąpień znaku
string tekst = "programowanie";
int liczbaA = 0;
foreach (char c in tekst)
    if (c == 'a') liczbaA++;

// Odwrócenie stringa
string odwrocony = new string(tekst.Reverse().ToArray());

// Sprawdzanie czy string jest liczbą
bool jestLiczba = int.TryParse("123", out _); // true

// Usunięcie duplikatów znaków
string unikalny = new string(tekst.Distinct().ToArray());

// Zliczanie słów
string zdanie = "To jest przykładowe zdanie";
int liczbaSłow = zdanie.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
```

7. Metody

7.1. Tworzenie metod

Metoda to nazwany blok kodu, który można wywoływać wielokrotnie. Schemat deklaracji:

```
modyfikator typ_zwracany NazwaMetody(parametry) { ciało }
```

```
// Metoda bez parametrów i bez zwracania wartości
static void Przywitaj()
{
    Console.WriteLine("Witaj!");
}

// Metoda z parametrami
static void Powitaj(string imie, int wiek)
{
    Console.WriteLine($"Cześć {imie}, masz {wiek} lat");
}

// Wywołanie
Przywitaj();
Powitaj("Jan", 25);
```

7.2. Parametry

```
// Parametry z wartościami domyślnymi
static void Wyswietl(string tekst, int ile = 1, char separator = '-')
{
    for (int i = 0; i < ile; i++)
        Console.WriteLine(tekst + separator);
}

Wyswietl("Witaj");           // Witaj- (domyślne wartości)
Wyswietl("Cześć", 3);        // Cześć- (trzy razy)
Wyswietl("Hej", 2, '!');     // Hej! (dwa razy)

// Argumenty nazwane – kolejność nie musi zgadzać się z deklaracją
Wyswietl(tekst: "Test", separator: '*', ile: 2);
```

7.3. Zwracanie wartości

```
// Zwracanie pojedynczej wartości
static int Suma(int a, int b)
{
    return a + b;
}

// Wyrażeniowa forma (expression body) – dla prostych metod
static int Kwadrat(int x) => x * x;
```

```

static string Powitanie(string imie) => $"Cześć, {imie}!";

// Zwracanie krotki (tuple) – kilka wartości naraz
static (int min, int max) MinMax(int[] tab)
{
    return (tab.Min(), tab.Max());
}

// Użycie
int s = Suma(3, 5); // 8
var (min, max) = MinMax(new[] {5, 3, 8, 1});
Console.WriteLine($"Min: {min}, Max: {max}");

```

7.4. ref i out

ref — parametr przekazywany przez referencję. Musi być zainicjalizowany przed wywołaniem:

```

static void Podwoj(ref int x)
{
    x *= 2;
}

int liczba = 5;
Podwoj(ref liczba);
Console.WriteLine(liczba); // 10

// Zamiana dwóch zmiennych
static void Zamien(ref int a, ref int b)
{
    int temp = a;
    a = b;
    b = temp;
}

```

out — podobny do **ref**, ale zmienna nie musi być zainicjalizowana. Metoda musi przypisać wartość:

```

static bool Podziel(int a, int b, out double wynik)
{
    if (b == 0) { wynik = 0; return false; }
    wynik = (double)a / b;
    return true;
}

if (Podziel(10, 3, out double rezultat))
    Console.WriteLine($"Wynik: {rezultat:F2}");

```

7.5. params

params pozwala przekazać zmienną liczbę argumentów:

```
static int Suma(params int[] liczby)
{
    int suma = 0;
    foreach (int l in liczby)
        suma += l;
    return suma;
}

Console.WriteLine(Suma(1, 2, 3));           // 6
Console.WriteLine(Suma(1, 2, 3, 4, 5));    // 15
Console.WriteLine(Suma());                 // 0
```

7.6. Przeciążanie metod

Kilka metod o tej samej nazwie, ale różnych parametrach:

```
static int Dodaj(int a, int b) ⇒ a + b;
static double Dodaj(double a, double b) ⇒ a + b;
static string Dodaj(string a, string b) ⇒ a + b;
static int Dodaj(int a, int b, int c) ⇒ a + b + c;

// Kompilator wybiera odpowiednią wersję
Console.WriteLine(Dodaj(3, 5));           // 8 (int)
Console.WriteLine(Dodaj(3.0, 5.5));       // 8.5 (double)
Console.WriteLine(Dodaj("Ala", "Jan"));   // AlaJan (string)
Console.WriteLine(Dodaj(1, 2, 3));        // 6 (int,int,int)
```

8. Operacje na plikach

Wymagają `using System.IO;`.

8.1. File

Klasa statyczna do prostych operacji na plikach:

```
using System.IO;

// Zapis
File.WriteAllText("plik.txt", "Witaj, świecie!");
File.WriteAllLines("lista.txt", new[] { "Linia 1", "Linia 2", "Linia 3" });

// Dopisywanie
File.AppendAllText("plik.txt", "\nDodatkowa linia");

// Odczyt
string zawartosc = File.ReadAllText("plik.txt");
string[] linie = File.ReadAllLines("lista.txt");

// Sprawdzanie
bool istnieje = File.Exists("plik.txt");
```

```
// Kopiowanie, przenoszenie, usuwanie
File.Copy("plik.txt", "kopia.txt");
File.Move("plik.txt", "nowy.txt");
File.Delete("kopia.txt");
```

8.2. FileInfo

Obiektowe API do informacji o pliku:

```
FileInfo fi = new FileInfo("plik.txt");
Console.WriteLine(fi.Name);           // plik.txt
Console.WriteLine(fi.FullName);       // pełna ścieżka
Console.WriteLine(fi.Length);         // rozmiar w bajtach
Console.WriteLine(fi.Extension);      // .txt
Console.WriteLine(fi.CreationTime);   // data utworzenia
Console.WriteLine(fi.LastWriteTime);  // data modyfikacji
Console.WriteLine(fi.Exists);         // czy istnieje

fi.CopyTo("kopia.txt");
fi.Delete();
```

8.3. Directory

```
// Tworzenie katalogu
Directory.CreateDirectory("MojFolder");

// Listowanie
string[] pliki = Directory.GetFiles(".", "*.txt");
string[] podkatalogi = Directory.GetDirectories(".");

// Sprawdzanie
bool istnieje = Directory.Exists("MojFolder");

// Usuwanie
Directory.Delete("MojFolder", recursive: true);

// Bieżący katalog
string biezacy = Directory.GetCurrentDirectory();
```

8.4. StreamReader

StreamReader umożliwia wydajny odczyt pliku tekstowego — szczególnie przydatny przy dużych plikach:

```
using (StreamReader sr = new StreamReader("plik.txt", System.Text.Encoding.UTF8))
{
    // Czytanie linijka po linijce
    string linia;
    while ((linia = sr.ReadLine()) != null)
    {
        Console.WriteLine(linia);
    }
}
```

```

    }
} // automatyczne zamknięcie pliku

// Alternatywnie – odczyt całości
using (StreamReader sr = new StreamReader("plik.txt"))
{
    string calosc = sr.ReadToEnd();
    Console.WriteLine(calosc);
}

```

8.5. StreamWriter

```

// Tworzenie nowego pliku (lub nadpisanie)
using (StreamWriter sw = new StreamWriter("plik.txt"))
{
    sw.WriteLine("Linia 1");
    sw.WriteLine("Linia 2");
    sw.Write("Bez nowej linii");
}

// Dopisywanie – append = true
using (StreamWriter sw = new StreamWriter("plik.txt", append: true))
{
    sw.WriteLine("Dodana linia");
}

// Z kodowaniem UTF-8
using (StreamWriter sw = new StreamWriter("plik.txt", false, System.Text.Encoding.UTF8))
{
    sw.WriteLine("Tekst z polskimi znakami: ąęłóśźżćń");
}

```

8.6. CSV

```

// Zapis CSV
using (StreamWriter sw = new StreamWriter("dane.csv", false, System.Text.Encoding.UTF8))
{
    sw.WriteLine("Imię;Nazwisko;Wiek");
    sw.WriteLine("Jan;Kowalski;25");
    sw.WriteLine("Anna;Nowak;30");
}

// Odczyt CSV
using (StreamReader sr = new StreamReader("dane.csv"))
{
    string naglowek = sr.ReadLine(); // pomiń nagłówek
    string linia;
    while ((linia = sr.ReadLine()) != null)
    {
        string[] pola = linia.Split(';');
        string imie = pola[0];
    }
}

```

```

        string nazwisko = pola[1];
        int wiek = int.Parse(pola[2]);
        Console.WriteLine($"{imie} {nazwisko}, lat {wiek}");
    }
}

```

8.7. JSON

W .NET 5+ dostępny jest wbudowany `System.Text.Json`. Starszą alternatywą jest biblioteka `Newtonsoft.Json`.

```

using System.Text.Json;

// Klasa do serializacji
class Osoba
{
    public string Imie { get; set; }
    public int Wiek { get; set; }
}

// Serializacja (obiekt → JSON string)
Osoba o = new Osoba { Imie = "Jan", Wiek = 25 };
string json = JsonSerializer.Serialize(o);
// {"Imie":"Jan","Wiek":25}

// Deserializacja (JSON string → obiekt)
Osoba odczytany = JsonSerializer.Deserialize<Osoba>(json);
Console.WriteLine(odczytany.Imie); // Jan

// Zapis do pliku
File.WriteAllText("osoba.json", json);

// Odczyt z pliku
string zawartosc = File.ReadAllText("osoba.json");
Osoba z_pliku = JsonSerializer.Deserialize<Osoba>(zawartosc);

```

9. Obsługa wyjątków

9.1. try / catch / finally

Wyjątek to błąd, który wystąpił podczas działania programu. `try-catch` pozwala go obsłużyć zamiast dopuścić do awarii:

```

try
{
    // Kod, który może rzucić wyjątek
    Console.Write("Podaj liczbę: ");
    int liczba = int.Parse(Console.ReadLine());
    int wynik = 100 / liczba;
    Console.WriteLine($"{100 / {liczba}} = {wynik}");
}
catch (FormatException)
{

```

```

// Zła format danych (np. wpisano "abc" zamiast liczby)
Console.WriteLine("Błąd: to nie jest liczba!");
}
catch (DivideByZeroException)
{
    // Dzielenie przez zero
    Console.WriteLine("Błąd: nie można dzielić przez zero!");
}
catch (Exception ex)
{
    // Dowolny inny wyjątek – ex.Message zawiera opis błędu
    Console.WriteLine($"Nieznany błąd: {ex.Message}");
}
finally
{
    // Wykona się zawsze – niezależnie od błędu
    Console.WriteLine("Koniec programu.");
}

```

Najczęstsze typy wyjątków w C#:

TYP WYJĄTKU	KIEDY WYSTĄPI
<code>FormatException</code>	Błąd konwersji np. <code>int.Parse("abc")</code>
<code>DivideByZeroException</code>	Dzielenie przez zero
<code>NullReferenceException</code>	Wywołanie metody na <code>null</code>
<code>IndexOutOfRangeException</code>	Indeks poza zakresem tablicy
<code>ArgumentException</code>	Nieprawidłowy argument metody
<code>FileNotFoundException</code>	Nie znaleziono pliku
<code>IOException</code>	Błąd wejścia/wyjścia
<code>OverflowException</code>	Przekroczenie zakresu typu
<code>InvalidCastException</code>	Nieprawidłowe rzutowanie

9.2. Własne wyjątki

```

// Definicja własnej klasy wyjątku
class BładWalutacji : Exception
{
    public BładWalutacji(string komunikat) : base(komunikat) { }
}

// Rzucanie własnego wyjątku
static void SprawdzWiek(int wiek)
{
    if (wiek < 0 || wiek > 150)
        throw new BładWalutacji($"Nieprawidłowy wiek: {wiek}");
}

// Użycie

```



```
try
{
    SprawdzWiek(-5);
}
catch (BładWalutacji ex)
{
    Console.WriteLine($"Bład: {ex.Message}");
}
```

10. Klasy i obiekty

10.1. Definicja klasy

Klasa to szablon (przepis) opisujący dane i zachowania obiektu:

```
class Samochod
{
    // Pola (dane)
    public string Marka;
    public string Model;
    public int RokProdukcji;

    // Metody (zachowania)
    public void Jedz()
    {
        Console.WriteLine($"{Marka} {Model} jedzie!");
    }

    public string OpisAuta()
    {
        return $"{Marka} {Model} ({RokProdukcji})";
    }
}
```

10.2. Obiekt

Obiekt to konkretna **instancja** klasy, tworzona operatorem `new`:

```
// Tworzenie obiektów
Samochod auto1 = new Samochod();
auto1.Marka = "Toyota";
auto1.Model = "Corolla";
auto1.RokProdukcji = 2020;

Samochod auto2 = new Samochod();
auto2.Marka = "Ford";
auto2.Model = "Focus";
auto2.RokProdukcji = 2018;

// Użycie
```

```
auto1.Jedz();  
Console.WriteLine(auto2.OpisAuta());
```

10.3. Pola

Pola to zmienne przechowujące stan obiektu. Mogą być publiczne (`public`) lub prywatne (`private`):

```
class Punkt  
{  
    public double X;          // pole publiczne  
    private double y;        // pole prywatne – dostępne tylko wewnątrz klasy  
    public static int Licznik = 0; // pole statyczne – wspólne dla wszystkich instancji  
}
```

10.4. Właściwości

Właściwości (properties) to bezpieczniejszy sposób udostępniania pól — z kontrolą odczytu i zapisu:

```
class Pracownik  
{  
    // Właściwość auto-implementowana  
    public string Imie { get; set; }  
  
    // Właściwość tylko do odczytu  
    public string Nazwisko { get; }  
  
    // Właściwość z walidacją  
    private int wiek;  
    public int Wiek  
    {  
        get { return wiek; }  
        set  
        {  
            if (value < 0 || value > 120)  
                throw new ArgumentException("Nieprawidłowy wiek");  
            wiek = value;  
        }  
    }  
  
    // Właściwość obliczana – bez pola bazowego  
    public string PelneImie => $"{Imie} {Nazwisko}";  
}
```

10.5. Konstruktory

Konstruktor to specjalna metoda wywoływana przy tworzeniu obiektu:

```
class Student  
{  
    public string Imie { get; set; }  
    public string Nazwisko { get; set; }  
    public double Srednia { get; set; }  
}
```

```

// Konstruktor domyślny (bez parametrów)
public Student()
{
    Srednia = 0.0;
}

// Konstruktor z parametrami
public Student(string imie, string nazwisko)
{
    Imie = imie;
    Nazwisko = nazwisko;
    Srednia = 0.0;
}

// Pełny konstruktor
public Student(string imie, string nazwisko, double srednia)
{
    Imie = imie;
    Nazwisko = nazwisko;
    Srednia = srednia;
}

public override string ToString()
{
    return $"{Imie} {Nazwisko} – średnia: {Srednia:F2}";
}
}

// Użycie
Student s1 = new Student();
Student s2 = new Student("Jan", "Kowalski");
Student s3 = new Student("Anna", "Nowak", 4.8);
Console.WriteLine(s3);    // Anna Nowak – średnia: 4,80

```

11. Hermetyzacja

Hermetyzacja (encapsulation) to ukrywanie wewnętrznych szczegółów klasy przed zewnętrznym kodem.

11.1. Modyfikatory dostępu

MODYFIKATOR	DOSTĘP
<code>public</code>	Wszędzie — z każdej klasy i przestrzeni nazw
<code>private</code>	Tylko wewnątrz tej samej klasy
<code>protected</code>	Wewnątrz klasy i klas pochodnych (dziedziczących)
<code>internal</code>	Wewnątrz tego samego projektu (assembly)
<code>protected internal</code>	<code>protected</code> LUB <code>internal</code>
<code>private protected</code>	<code>protected</code> ORAZ <code>internal</code>

```

class KontoBankowe
{
    private decimal saldo;           // ukryte – nie dostępne z zewnątrz
    public string NumerKonta { get; } // dostępne z zewnątrz

    public KontoBankowe(string numer, decimal saldoPoczkowe)
    {
        NumerKonta = numer;
        saldo = saldoPoczkowe;
    }

    public void Wplata(decimal kwota)
    {
        if (kwota ≤ 0) throw new ArgumentException("Kwota musi być dodatnia");
        saldo += kwota;
    }

    public bool Wypłata(decimal kwota)
    {
        if (kwota > saldo) return false;
        saldo -= kwota;
        return true;
    }

    public decimal PodajSaldo() ⇒ saldo;
}

```

11.2. get i set

```

class Temperatura
{
    private double celsjusz;

    // get – odczyt; set – zapis
    public double Celsjusz
    {
        get { return celsjusz; }
        set
        {
            if (value < -273.15)
                throw new ArgumentException("Poniżej zera absolutnego!");
            celsjusz = value;
        }
    }

    // Właściwość tylko do odczytu (brak set)
    public double Fahrenheit
    {
        get { return celsjusz * 9.0 / 5.0 + 32; }
    }
}

```

```
// Właściwość tylko do zapisu (rzadkie)
public double UstawKelvin
{
    set { celsjusz = value - 273.15; }
}

// Auto-property – skrót gdy nie potrzebujemy walidacji
public string Opis { get; set; } = "Temperatura";

// Init-only property – ustawiana tylko w konstruktorze lub object initializer (C# 9+)
public DateTime DataPomiaru { get; init; }
}
```

12. Dziedziczenie

12.1. Klasy bazowe i pochodne

Dziedziczenie pozwala tworzyć nowe klasy na podstawie istniejących:

```
// Klasa bazowa
class Zwierze
{
    public string Imie { get; set; }
    public int Wiek { get; set; }

    public Zwierze(string imie, int wiek)
    {
        Imie = imie;
        Wiek = wiek;
    }

    public virtual void Wydaj_dzwiek()
    {
        Console.WriteLine("Jakiś dźwięk...");
    }

    public void Oddychaj()
    {
        Console.WriteLine($"{Imie} oddycha");
    }
}

// Klasa pochodna – dziedziczy z Zwierze
class Pies : Zwierze
{
    public string Rasa { get; set; }

    public Pies(string imie, int wiek, string rasa) : base(imie, wiek)
    {
        Rasa = rasa;
    }
}
```

```

// Nadpisanie metody wirtualnej
public override void Wydaj_dzwiek()
{
    Console.WriteLine($"{Imie} szczeka: HAU HAU!");
}

// Nowa metoda – tylko w klasie Pies
public void Aportuj()
{
    Console.WriteLine($"{Imie} aportuje!");
}
}

```

12.2. override i virtual

- **virtual** — w klasie bazowej oznacza, że metoda MOŻE być nadpisana
- **override** — w klasie pochodnej oznacza nadpisanie metody bazowej

```

class Kształt
{
    public virtual double ObliczPole() => 0;
    public virtual string NazwaKształtu() => "Kształt";
}

class Kolo : Kształt
{
    public double Promien { get; set; }
    public Kolo(double promien) { Promien = promien; }

    public override double ObliczPole() => Math.PI * Promien * Promien;
    public override string NazwaKształtu() => "Koło";
}

class Prostokat : Kształt
{
    public double Szerokosc { get; set; }
    public double Wysokosc { get; set; }

    public Prostokat(double szerokosc, double wysokosc)
    {
        Szerokosc = szerokosc;
        Wysokosc = wysokosc;
    }

    public override double ObliczPole() => Szerokosc * Wysokosc;
    public override string NazwaKształtu() => "Prostokąt";
}

```

12.3. base

- **base** — odwołuje się do klasy bazowej — jej konstruktora lub metod:

```

class Pracownik : Osoba
{
    public string Stanowisko { get; set; }

    // Wywołanie konstruktora klasy bazowej
    public Pracownik(string imie, string nazwisko, string stanowisko)
        : base(imie, nazwisko)
    {
        Stanowisko = stanowisko;
    }

    public override string ToString()
    {
        // Wywołanie metody bazowej
        string bazowy = base.ToString();
        return $"{bazowy} – {Stanowisko}";
    }
}

```

13. Polimorfizm

Polimorfizm oznacza, że obiekty różnych klas mogą być traktowane jak obiekty klasy bazowej, a wywoływane metody będą zachowywały się odpowiednio do rzeczywistego typu.

13.1. Polimorfizm metod

```

// Polimorfizm przez dziedziczenie
Kształt[] kształty = new Kształt[]
{
    new Koło(5),
    new Prostokat(4, 6),
    new Koło(3)
};

foreach (Kształt k in kształty)
{
    // Wywołuje odpowiednią wersję ObliczPole() dla każdego kształtu
    Console.WriteLine($"{k.NazwaKształtu():} - {k.ObliczPole():F2}");
}

// Koło: 78,54
// Prostokąt: 24,00
// Koło: 28,27

```

13.2. Przeciążanie

Opisane w rozdziale 7.6 — wiele metod o tej samej nazwie, różnych parametrach.

13.3. Nadpisywanie

Nadpisywanie (overriding) to zmiana zachowania metody z klasy bazowej w klasie pochodnej przy użyciu `virtual` + `override`.

Jeśli **nie** chcemy pozwolić na dalsze nadpisywanie, używamy `sealed`:

```
class KołoPrzekresle : Koło
{
    public sealed override double ObliczPole()
    {
        return base.ObliczPole() / 2;
    }

    // Klasy dziedziczące z KołoPrzekresle nie mogą już nadpisać ObliczPole
}
```

14. Klasy abstrakcyjne i interfejsy

14.1. abstract

Klasa abstrakcyjna **nie może być instancjonowana** — służy jako szablon dla klas pochodnych. Może zawierać zarówno metody abstrakcyjne (bez implementacji) jak i zwykłe:

```
abstract class Figura
{
    // Właściwości wspólne
    public string Kolor { get; set; } = "czarny";

    // Metoda abstrakcyjna — klasa pochodna MUSI ją zaimplementować
    public abstract double ObliczPole();
    public abstract double ObliczObwod();

    // Metoda zwykła — klasa pochodna MOŻE ją nadpisać
    public virtual void Wyświetl()
    {
        Console.WriteLine($"Figura [{Kolor}]: pole={ObliczPole():F2}, obwód={ObliczObwod():F2}");
    }
}

class Trojkat : Figura
{
    public double A { get; set; }
    public double B { get; set; }
    public double C { get; set; }

    public Trojkat(double a, double b, double c) { A = a; B = b; C = c; }

    public override double ObliczObwod() => A + B + C;

    public override double ObliczPole()
    {
        double s = ObliczObwod() / 2;
        return Math.Sqrt(s * (s - A) * (s - B) * (s - C)); // wzór Herona
    }
}
```


14.2. interface

Interfejs to kontrakt — klasa implementująca go musi dostarczyć wszystkie wymienione metody i właściwości. Interfejs nie zawiera implementacji (chyba że są to metody domyślne — C# 8.0+):

```
// Interfejsy – nazwy zaczynają się od "I" z konwencji
interface IDrukowalne
{
    void Drukuj();
    string GenerujRaport();
}

interface IZapisywalne
{
    void Zapisz(string sciezka);
    bool Wczytaj(string sciezka);
}
```

14.3. Implementacja interfejsów

Klasa może implementować wiele interfejsów (w przeciwieństwie do dziedziczenia — tylko jedna klasa bazowa):

```
class Dokument : IDrukowalne, IZapisywalne
{
    public string Tytuł { get; set; }
    public string Tresc { get; set; }

    // Implementacja IDrukowalne
    public void Drukuj()
    {
        Console.WriteLine($"=== {Tytuł} ===");
        Console.WriteLine(Tresc);
    }

    public string GenerujRaport() => $"Dokument: {Tytuł} ({Tresc.Length} znaków)";

    // Implementacja IZapisywalne
    public void Zapisz(string sciezka)
    {
        File.WriteAllText(sciezka, $"{Tytuł}\n{Tresc}");
    }

    public bool Wczytaj(string sciezka)
    {
        if (!File.Exists(sciezka)) return false;
        string[] linie = File.ReadAllLines(sciezka);
        Tytuł = linie[0];
        Tresc = string.Join("\n", linie[1..]);
        return true;
    }
}

// Polimorfizm przez interfejsy
IDrukowalne[] drukowalne = { new Dokument { Tytuł = "Test", Tresc = "Treść" } };
```

```
foreach (IDrukowalne d in drukowalne)
    d.Drukuj();
```

15. LINQ

LINQ (Language Integrated Query) pozwala odpytywać kolekcje w stylu SQL. Wymaga `using System.Linq;`.

15.1. Where

Filtrowanie — zwraca elementy spełniające warunek:

```
int[] liczby = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

// Syntax metod (Method Syntax) – zalecany
var parzyste = liczby.Where(x => x % 2 == 0);
// { 2, 4, 6, 8, 10 }

List<string> imiona = new List<string> { "Jan", "Anna", "Adam", "Beata", "Aleksander" };
var dlugie = imiona.Where(s => s.Length > 4);
// { "Anna", "Beata", "Aleksander" }
```

15.2. Select

Projekcja — przekształca każdy element:

```
int[] liczby = { 1, 2, 3, 4, 5 };
var kwadraty = liczby.Select(x => x * x);
// { 1, 4, 9, 16, 25 }

var duze = imiona.Select(s => s.ToUpper());
// { "JAN", "ANNA", ... }

// Projekcja na anonimowy obiekt
var dane = imiona.Select(s => new { Imie = s, Dlugosc = s.Length });
foreach (var d in dane)
    Console.WriteLine($"{d.Imie}: {d.Dlugosc} znaków");
```

15.3. OrderBy i OrderByDescending

```
int[] liczby = { 5, 2, 8, 1, 9, 3 };
var rosnaco = liczby.OrderBy(x => x);           // { 1, 2, 3, 5, 8, 9 }
var malejaco = liczby.OrderByDescending(x => x); // { 9, 8, 5, 3, 2, 1 }

List<string> imiona = new List<string> { "Jan", "Anna", "Beata" };
var alfabetycznie = imiona.OrderBy(s => s);
var po_dlugosci = imiona.OrderBy(s => s.Length).ThenBy(s => s); // sortowanie po kilku kryteriach
```

15.4. First i FirstOrDefault

```
int[] liczby = { 5, 2, 8, 1, 9, 3 };

int pierwszy = liczby.First();           // 5
int pierwszyParzysty = liczby.First(x => x % 2 == 0); // 2

// FirstOrDefault – zwraca default(T) zamiast wyjątku gdy nic nie znaleziono
int? wynik = liczby.FirstOrDefault(x => x > 100); // 0 (domyślna wartość int)
string s = imiona.FirstOrDefault(x => x.StartsWith("Z")); // null

// Analogicznie: Last, LastOrDefault, Single, SingleOrDefault
```

15.5. Any i All

```
int[] liczby = { 1, 2, 3, 4, 5 };

bool czyJestParzysty = liczby.Any(x => x % 2 == 0); // true
bool czyWszystkieDodadnie = liczby.All(x => x > 0); // true
bool czyWszystkieDuze = liczby.All(x => x > 3); // false
bool czyPusta = liczby.Any(); // true (niepusta)
```

15.6. GroupBy

Grupowanie elementów według klucza:

```
string[] slowa = { "ala", "kot", "pies", "ant", "byk", "kos" };

var grupy = slowa.GroupBy(s => s.Length);
foreach (var grupa in grupy)
{
    Console.WriteLine($"Długość {grupa.Key}: ");
    Console.WriteLine(string.Join(", ", grupa));
}

// Długość 3: ala, kot, ant, byk, kos
// Długość 4: pies

// Przykład z obiektami
class Ocena { public string Przedmiot; public int Wartosc; }
var oceny = new List<Ocena> { /* ... */ };
var wgPrzedmiotu = oceny.GroupBy(o => o.Przedmiot)
    .Select(g => new { Przedmiot = g.Key, Srednia = g.Average(o => o.Wartosc) });
```

Przydatne metody agregujące LINQ:

METODA	OPIS
<code>.Count()</code> / <code>.Count(predicate)</code>	Liczba elementów
<code>.Sum(x => x.Pole)</code>	Suma
<code>.Average(x => x.Pole)</code>	Średnia
<code>.Min(x => x.Pole)</code>	Minimum
<code>.Max(x => x.Pole)</code>	Maksimum
<code>.ToList()</code>	Konwersja na <code>List<T></code>
<code>.ToArray()</code>	Konwersja na tablicę
<code>.Distinct()</code>	Unikalne elementy
<code>.Take(n)</code>	Pierwsze n elementów
<code>.Skip(n)</code>	Pomiń n elementów

16. Sortowanie

16.1. Bubble Sort

Porównuje sąsiednie elementy i zamienia je miejscami jeśli są w złej kolejności. Powtarza do całkowitego posortowania.

```
static void BubbleSort(int[] tab)
{
    int n = tab.Length;
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (tab[j] > tab[j + 1])
            {
                // Zamiana
                int temp = tab[j];
                tab[j] = tab[j + 1];
                tab[j + 1] = temp;
            }
        }
    }
}
```

16.2. Selection Sort

Znajdowanie minimum w nieposortowanej części i wstawianie go na właściwą pozycję.

```
static void SelectionSort(int[] tab)
{
    int n = tab.Length;
    for (int i = 0; i < n - 1; i++)
    {
        int minIdx = i;
```

```

        for (int j = i + 1; j < n; j++)
        {
            if (tab[j] < tab[minIdx])
                minIdx = j;
        }
        // Zamiana elementu minimalnego z pierwszym nieposortowanym
        int temp = tab[minIdx];
        tab[minIdx] = tab[i];
        tab[i] = temp;
    }
}

```

16.3. Insertion Sort

Buduje posortowaną część tablicy, wstawiając każdy nowy element na właściwe miejsce.

```

static void InsertionSort(int[] tab)
{
    int n = tab.Length;
    for (int i = 1; i < n; i++)
    {
        int klucz = tab[i];
        int j = i - 1;
        while (j ≥ 0 && tab[j] > klucz)
        {
            tab[j + 1] = tab[j];
            j--;
        }
        tab[j + 1] = klucz;
    }
}

```

16.4. Merge Sort

Dziel i zwyciężaj — rekurencyjny podział na połowy, a następnie scalanie posortowanych części.

```

static void MergeSort(int[] tab, int lewy, int prawy)
{
    if (lewy < prawy)
    {
        int srodek = (lewy + prawy) / 2;
        MergeSort(tab, lewy, srodek);
        MergeSort(tab, srodek + 1, prawy);
        Scalaj(tab, lewy, srodek, prawy);
    }
}

static void Scalaj(int[] tab, int lewy, int srodek, int prawy)
{
    int n1 = srodek - lewy + 1;
    int n2 = prawy - srodek;
    int[] L = new int[n1];
    int[] R = new int[n2];

```

```

Array.Copy(tab, lewy, L, 0, n1);
Array.Copy(tab, srodek + 1, R, 0, n2);

int i = 0, j = 0, k = lewy;
while (i < n1 && j < n2)
{
    if (L[i] ≤ R[j]) tab[k++] = L[i++];
    else tab[k++] = R[j++];
}

while (i < n1) tab[k++] = L[i++];
while (j < n2) tab[k++] = R[j++];
}

// Wywołanie:
// MergeSort(tab, 0, tab.Length - 1);

```

16.5. Quick Sort

Wybiera element pivot i partycjonuje tablicę — elementy mniejsze od pivota idą w lewo, większe w prawo.

```

static void QuickSort(int[] tab, int lewy, int prawy)
{
    if (lewy < prawy)
    {
        int p = Partycja(tab, lewy, prawy);
        QuickSort(tab, lewy, p - 1);
        QuickSort(tab, p + 1, prawy);
    }
}

static int Partycja(int[] tab, int lewy, int prawy)
{
    int pivot = tab[prawy];
    int i = lewy - 1;
    for (int j = lewy; j < prawy; j++)
    {
        if (tab[j] ≤ pivot)
        {
            i++;
            (tab[i], tab[j]) = (tab[j], tab[i]); // zamiana z dekonstrukcją krotki
        }
    }
    (tab[i + 1], tab[prawy]) = (tab[prawy], tab[i + 1]);
    return i + 1;
}

// Wywołanie:
// QuickSort(tab, 0, tab.Length - 1);

```

17. Wyszukiwanie

17.1. Liniowe

Przegląda tablicę od początku do końca. Działa na nieposortowanych danych. Złożoność: $O(n)$.

```
static int SzukajLinearnie(int[] tab, int szukana)
{
    for (int i = 0; i < tab.Length; i++)
    {
        if (tab[i] == szukana)
            return i;    // zwraca indeks
    }
    return -1;           // nie znaleziono
}

// Użycie
int[] tab = { 5, 3, 8, 1, 9, 2 };
int indeks = SzukajLinearnie(tab, 8);    // 2
int brak = SzukajLinearnie(tab, 100);    // -1
```

17.2. Binarne

Wymaga **posortowanej** tablicy. Dzieli zakres poszukiwań na pół przy każdej iteracji. Złożoność: $O(\log n)$.

```
static int SzukajBinarnie(int[] tab, int szukana)
{
    int lewy = 0;
    int prawy = tab.Length - 1;

    while (lewy <= prawy)
    {
        int srodek = (lewy + prawy) / 2;

        if (tab[srodek] == szukana)
            return srodek;    // znaleziono
        else if (tab[srodek] < szukana)
            lewy = srodek + 1;    // szukaj w prawej połowie
        else
            prawy = srodek - 1;    // szukaj w lewej połowie
    }
    return -1;    // nie znaleziono
}

// Wbudowana metoda .NET (tablica musi być posortowana)
int[] tab = { 1, 3, 5, 7, 9, 11 };
int idx = Array.BinarySearch(tab, 7);    // 3
```

18. Algorytmy matematyczne

18.1. NWD

Największy Wspólny Dzielnik — algorytm Euklidesa:

```
static int NWD(int a, int b)
{
    while (b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

Console.WriteLine(NWD(48, 18));    // 6
Console.WriteLine(NWD(100, 75));  // 25
```

18.2. NWW

Najmniejsza Wspólna Wielokrotność:

```
static int NWW(int a, int b)
{
    return a / NWD(a, b) * b;    // kolejność ważna – unikamy przepełnienia
}

Console.WriteLine(NWW(4, 6));    // 12
Console.WriteLine(NWW(3, 5));    // 15
```

18.3. Liczby pierwsze

```
static bool CzyPierwsza(int n)
{
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;

    for (int i = 3; i ≤ Math.Sqrt(n); i += 2)
    {
        if (n % i == 0) return false;
    }
    return true;
}

Console.WriteLine(CzyPierwsza(7));    // True
Console.WriteLine(CzyPierwsza(10));   // False
Console.WriteLine(CzyPierwsza(2));    // True
```


18.4. Sito Eratostenesa

Efektywne wyznaczanie wszystkich liczb pierwszych do n:

```
static bool[] SitoEratostenesa(int n)
{
    bool[] czyPierwsza = new bool[n + 1];
    // Inicjalizacja – zakładamy, że wszystkie są pierwsze
    for (int i = 2; i ≤ n; i++)
        czyPierwsza[i] = true;

    for (int i = 2; i * i ≤ n; i++)
    {
        if (czyPierwsza[i])
        {
            // Oznacz wielokrotności i jako złożone
            for (int j = i * i; j ≤ n; j += i)
                czyPierwsza[j] = false;
        }
    }
    return czyPierwsza;
}

// Wypisz wszystkie pierwsze do 50
bool[] sito = SitoEratostenesa(50);
for (int i = 2; i ≤ 50; i++)
    if (sito[i]) Console.Write(i + " ");
// 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

18.5. Rozkład na czynniki pierwsze

```
static List<int> RozkladNaCzynniki(int n)
{
    List<int> czynniki = new List<int>();
    for (int d = 2; d * d ≤ n; d++)
    {
        while (n % d == 0)
        {
            czynniki.Add(d);
            n /= d;
        }
    }
    if (n > 1) czynniki.Add(n);
    return czynniki;
}

var czynniki = RozkladNaCzynniki(360);
Console.WriteLine(string.Join(" × ", czynniki));
// 2 × 2 × 2 × 3 × 3 × 5
```

19. Algorytmy tekstowe

19.1. Palindrom

Słowo lub zdanie, które czyta się tak samo od przodu i od tyłu:

```
static bool CzyPalindrom(string s)
{
    s = s.ToLower().Replace(" ", "");    // normalizacja
    int lewy = 0, prawy = s.Length - 1;
    while (lewy < prawy)
    {
        if (s[lewy] != s[prawy]) return false;
        lewy++;
        prawy--;
    }
    return true;
}

Console.WriteLine(CzyPalindrom("kajak"));    // True
Console.WriteLine(CzyPalindrom("Ala"));      // True
Console.WriteLine(CzyPalindrom("program"));  // False

// Wersja jednolinijkowa z LINQ
static bool CzyPalindromLinq(string s)
{
    s = s.ToLower().Replace(" ", "");
    return s == new string(s.Reverse().ToArray());
}
```

19.2. Anagram

Dwa słowa są anagramami jeśli zawierają dokładnie te same litery:

```

static bool CzyAnagram(string a, string b)
{
    a = a.ToLower().Replace(" ", "");
    b = b.ToLower().Replace(" ", "");

    if (a.Length != b.Length) return false;

    // Sortuj litery i porównaj
    char[] sortA = a.ToCharArray();
    char[] sortB = b.ToCharArray();
    Array.Sort(sortA);
    Array.Sort(sortB);

    return new string(sortA) == new string(sortB);
}

Console.WriteLine(CzyAnagram("listen", "silent")); // True
Console.WriteLine(CzyAnagram("hello", "world")); // False
Console.WriteLine(CzyAnagram("anagram", "nagaram")); // True

```

19.3. Zliczanie znaków

```

static Dictionary<char, int> ZliczZnaki(string tekst)
{
    Dictionary<char, int> licznik = new Dictionary<char, int>();
    foreach (char c in tekst.ToLower())
    {
        if (char.IsLetter(c))
        {
            if (licznik.ContainsKey(c))
                licznik[c]++;
            else
                licznik[c] = 1;
        }
    }
    return licznik;
}

// Użycie
string tekst = "programowanie";
var zliczenie = ZliczZnaki(tekst);
foreach (var para in zliczenie.OrderByDescending(p => p.Value))
    Console.WriteLine($"{para.Key}: {para.Value}");

```

19.4. Szyfr Cezara

Przesuwa każdą literę alfabetu o podaną liczbę pozycji:

```

static string SzyfrCezara(string tekst, int przesuniecie)
{
    przesuniecie = ((przesuniecie % 26) + 26) % 26; // obsługa ujemnych

```

```

        StringBuilder wynik = new StringBuilder();

        foreach (char c in tekst)
        {
            if (char.IsLetter(c))
            {
                char baza = char.IsUpper(c) ? 'A' : 'a';
                char zaszyfrowany = (char)((c - baza + przesuniecie) % 26 + baza);
                wynik.Append(zaszyfrowany);
            }
            else
            {
                wynik.Append(c);    // znaki niebędące literami bez zmian
            }
        }

        return wynik.ToString();
    }

    static string OdszyfrujCezara(string tekst, int przesuniecie)
    {
        return SzyfrCezara(tekst, 26 - przesuniecie);
    }

    string tajny = SzyfrCezara("Hello World", 3);    // Kloor Zruog
    string jawny = OdszyfrujCezara(tajny, 3);        // Hello World

```

20. Tablice dwuwymiarowe

20.1. Tworzenie

```

// Deklaracja – tablica 3x4
int[,] macierz = new int[3, 4];

// Inicjalizacja z wartościami
int[,] init = {
    { 1, 2, 3, 4 },
    { 5, 6, 7, 8 },
    { 9, 10, 11, 12 }
};

// Dostęp
macierz[0, 0] = 10;    // wiersz 0, kolumna 0
Console.WriteLine(init[1, 2]);    // 7 – wiersz 1, kolumna 2

```

20.2. Iteracja

```

int wiersze = macierz.GetLength(0);
int kolumny = macierz.GetLength(1);

for (int i = 0; i < wiersze; i++)

```

```

{
    for (int j = 0; j < kolumny; j++)
    {
        Console.Write($"{init[i, j],4}");
    }
    Console.WriteLine();
}

// Wynik:
//   1   2   3   4
//   5   6   7   8
//   9  10  11  12

```

20.3. Suma przekątnych

```

static (int glowna, int poboczna) SumaPrzekatnych(int[,] m)
{
    int n = m.GetLength(0);
    int glowna = 0, poboczna = 0;

    for (int i = 0; i < n; i++)
    {
        glowna += m[i, i];           // przekątna główna
        poboczna += m[i, n - 1 - i]; // przekątna poboczna
    }
    return (glowna, poboczna);
}

int[,] kwadrat = {
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};

var (g, p) = SumaPrzekatnych(kwadrat);
Console.WriteLine($"Główna: {g}, Poboczna: {p}"); // Główna: 15, Poboczna: 15

```

20.4. Obrót macierzy

Obrót macierzy kwadratowej o 90° w prawo:

```

static int[,] ObrocMacierz(int[,] m)
{
    int n = m.GetLength(0);
    int[,] wynik = new int[n, n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            wynik[j, n - 1 - i] = m[i, j];

    return wynik;
}

int[,] przed = { { 1, 2, 3 }, { 4, 5, 6 }, { 7, 8, 9 } };

```

```
int[,] po = ObrocMacierz(przed);  
  
// 7 4 1  
// 8 5 2  
// 9 6 3
```

21. SQL dla INF.04

21.1. SELECT

```
-- Wszystkie rekordy z tabeli  
SELECT * FROM Uczniowie;  
  
-- Wybrane kolumny  
SELECT Imie, Nazwisko, Wiek FROM Uczniowie;  
  
-- Z aliasami kolumn  
SELECT Imie AS "Imię ucznia", Srednia AS "Ocena" FROM Uczniowie;  
  
-- Bez duplikatów  
SELECT DISTINCT Miasto FROM Uczniowie;
```

21.2. WHERE

```
-- Podstawowy warunek  
SELECT * FROM Uczniowie WHERE Wiek ≥ 18;  
  
-- Wiele warunków  
SELECT * FROM Uczniowie WHERE Wiek > 16 AND Srednia ≥ 4.0;  
SELECT * FROM Uczniowie WHERE Miasto = 'Warszawa' OR Miasto = 'Kraków';  
  
-- Wzorzec LIKE  
SELECT * FROM Uczniowie WHERE Nazwisko LIKE 'Kował%'; -- zaczyna się od Kował  
SELECT * FROM Uczniowie WHERE Imie LIKE '%an%'; -- zawiera "an"  
  
-- Zakres BETWEEN  
SELECT * FROM Produkty WHERE Cena BETWEEN 10 AND 100;  
  
-- Lista wartości IN  
SELECT * FROM Uczniowie WHERE Miasto IN ('Warszawa', 'Kraków', 'Gdańsk');  
  
-- Wartości NULL  
SELECT * FROM Pracownicy WHERE Telefon IS NULL;  
SELECT * FROM Pracownicy WHERE Email IS NOT NULL;
```

21.3. ORDER BY

```
-- Rosnąco (domyślnie)  
SELECT * FROM Uczniowie ORDER BY Nazwisko;
```

```

SELECT * FROM Uczniowie ORDER BY Srednia ASC;

-- Malejąco
SELECT * FROM Uczniowie ORDER BY Srednia DESC;

-- Po kilku kolumnach
SELECT * FROM Uczniowie ORDER BY Klasa ASC, Srednia DESC;

```

21.4. GROUP BY

```

-- Grupowanie z agregacją
SELECT Klasa, COUNT(*) AS LiczbaUczniow
FROM Uczniowie
GROUP BY Klasa;

SELECT Klasa, AVG(Srednia) AS SredniaKlasy, MAX(Srednia) AS Najlepsza
FROM Uczniowie
GROUP BY Klasa;

-- Filtrowanie grup – HAVING (nie WHERE!)
SELECT Klasa, COUNT(*) AS Liczba
FROM Uczniowie
GROUP BY Klasa
HAVING COUNT(*) > 20;

-- Funkcje agregujące
-- COUNT(*) – liczba wierszy
-- SUM(kolumna) – suma
-- AVG(kolumna) – średnia
-- MIN(kolumna) – minimum
-- MAX(kolumna) – maksimum

```

21.5. INSERT

```

-- Wstawianie jednego rekordu (z podaniem kolumn – zalecane)
INSERT INTO Uczniowie (Imie, Nazwisko, Wiek, Srednia)
VALUES ('Jan', 'Kowalski', 18, 4.5);

-- Wstawianie bez podania kolumn (kolejność musi zgadzać się ze schematem)
INSERT INTO Uczniowie VALUES (NULL, 'Anna', 'Nowak', 17, 4.0, 'Warszawa');

-- Wstawianie wielu rekordów
INSERT INTO Uczniowie (Imie, Nazwisko)
VALUES ('Piotr', 'Wiśniewski'),
       ('Maria', 'Dąbrowska'),
       ('Tomasz', 'Lewandowski');

```

21.6. UPDATE

```
-- ZAWSZE używaj WHERE – bez niego zmienisz WSZYSTKIE rekordy!
UPDATE Uczniowie
SET Srednia = 5.0
WHERE Id = 5;

-- Kilka kolumn naraz
UPDATE Uczniowie
SET Imie = 'Adam', Wiek = 19
WHERE Id = 10;

-- Wartość obliczona
UPDATE Produkty
SET Cena = Cena * 1.1 -- podwyżka o 10%
WHERE Kategoria = 'Elektronika';
```

21.7. DELETE

```
-- ZAWSZE używaj WHERE – bez niego usuniesz WSZYSTKIE rekordy!
DELETE FROM Uczniowie WHERE Id = 5;

-- Usunięcie według warunku
DELETE FROM Uczniowie WHERE Srednia < 2.0;

-- Usunięcie wszystkich (nie zalecane – lepiej TRUNCATE)
DELETE FROM Uczniowie;
```

21.8. JOIN

```
-- INNER JOIN – tylko pasujące rekordy w obu tabelach
SELECT u.Imie, u.Nazwisko, k.NazwaKlasy
FROM Uczniowie u
INNER JOIN Klasy k ON u.KlasaId = k.Id;

-- LEFT JOIN – wszystkie z lewej tabeli + pasujące z prawej
SELECT u.Imie, u.Nazwisko, o.Tytul
FROM Uczniowie u
LEFT JOIN Osiagniecia o ON u.Id = o.UczenId;

-- RIGHT JOIN – wszystkie z prawej + pasujące z lewej
SELECT u.Imie, k.NazwaKlasy
FROM Uczniowie u
RIGHT JOIN Klasy k ON u.KlasaId = k.Id;

-- Łączenie kilku tabel
SELECT u.Imie, u.Nazwisko, k.NazwaKlasy, s.Miasto
FROM Uczniowie u
JOIN Klasy k ON u.KlasaId = k.Id
```



```
JOIN Szkoły s ON k.SzkołaId = s.Id
WHERE s.Miasto = 'Warszawa';
```

22. SQL Server + C#

Wymagają `using System.Data.SqlClient;` (lub `Microsoft.Data.SqlClient`).

22.1. SqlConnection

```
// Connection string – dane połączenia z bazą
string connectionString = "Server=localhost;Database=Szkoła;User Id=sa;Password=hasło;";
// Dla Windows Authentication:
// string connectionString = "Server=localhost;Database=Szkoła;Trusted_Connection=True;";

// Tworzenie i otwieranie połączenia
using (SqlConnection conn = new SqlConnection(connectionString))
{
    conn.Open();
    Console.WriteLine("Połączono z bazą: " + conn.Database);
    // ... operacje na bazie ...
} // automatyczne zamknięcie po wyjściu z using
```

22.2. SqlCommand i SqlDataReader

```
string connectionString = "Server=localhost;Database=Szkoła;Trusted_Connection=True;";

using (SqlConnection conn = new SqlConnection(connectionString))
{
    conn.Open();

    // Zapytanie SELECT
    string sql = "SELECT Id, Imie, Nazwisko, Srednia FROM Uczniowie ORDER BY Nazwisko";
    using (SqlCommand cmd = new SqlCommand(sql, conn))
    {
        using (SqlDataReader reader = cmd.ExecuteReader())
        {
            while (reader.Read())
            {
                int id = reader.GetInt32(0); // kolumna 0
                string imie = reader.GetString(1); // kolumna 1
                string nazwisko = reader["Nazwisko"].ToString(); // po nazwie
                double srednia = reader.GetDouble(3);

                Console.WriteLine($"{id}. {imie} {nazwisko} – {srednia:F2}");
            }
        }
    }

    // INSERT / UPDATE / DELETE – ExecuteNonQuery
    string insert = "INSERT INTO Uczniowie (Imie, Nazwisko) VALUES ('Test', 'Testowy')";
```

```

using (SqlCommand cmd = new SqlCommand(insert, conn))
{
    int zmienione = cmd.ExecuteNonQuery();
    Console.WriteLine($"Dodano {zmienione} rekord(ów)");
}

// Zapytanie zwracające jedną wartość – ExecuteScalar
string count = "SELECT COUNT(*) FROM Uczniowie";
using (SqlCommand cmd = new SqlCommand(count, conn))
{
    int liczba = (int)cmd.ExecuteScalar();
    Console.WriteLine($"Liczba uczniów: {liczba}");
}
}

```

22.3. Parametryzowane zapytania

Parametryzowane zapytania chronią przed **SQL Injection** — zawsze ich używaj przy danych od użytkownika:

```

using (SqlConnection conn = new SqlConnection(connectionString))
{
    conn.Open();

    // Bezpieczne zapytanie z parametrami – zamiast string concatenation
    string sql = "SELECT * FROM Uczniowie WHERE Imie = @Imie AND Wiek ≥ @MinWiek";
    using (SqlCommand cmd = new SqlCommand(sql, conn))
    {
        // Dodawanie parametrów
        cmd.Parameters.AddWithValue("@Imie", "Jan");
        cmd.Parameters.AddWithValue("@MinWiek", 16);

        using (SqlDataReader reader = cmd.ExecuteReader())
        {
            while (reader.Read())
                Console.WriteLine($"{reader["Imie"]} {reader["Nazwisko"]}");
        }
    }

    // INSERT z parametrami
    string insert = @"INSERT INTO Uczniowie (Imie, Nazwisko, Wiek, Srednia)
        VALUES (@Imie, @Nazwisko, @Wiek, @Srednia)";
    using (SqlCommand cmd = new SqlCommand(insert, conn))
    {
        cmd.Parameters.AddWithValue("@Imie", "Anna");
        cmd.Parameters.AddWithValue("@Nazwisko", "Nowak");
        cmd.Parameters.AddWithValue("@Wiek", 18);
        cmd.Parameters.AddWithValue("@Srednia", 4.5);

        cmd.ExecuteNonQuery();
    }
}

```

23. Windows Forms

Windows Forms (WinForms) to framework do tworzenia aplikacji okienkowych w .NET.

23.1. Tworzenie formularza

Formularz tworzony jest w Visual Studio przez dodanie nowego projektu **Windows Forms App (.NET)**. Każdy formularz składa się z:

- Plik `.cs` — kod logiki (obsługa zdarzeń, metody)
- Plik `.Designer.cs` — automatycznie generowany kod układu kontroltek

```
// Program.cs — punkt wejścia aplikacji WinForms
using System;
using System.Windows.Forms;

static class Program
{
    [STAThread]
    static void Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new Form1()); // uruchom główny formularz
    }
}
```

```
// Form1.cs — logika formularza
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent(); // inicjalizacja kontroltek z Designer
        // Dodatkowy kod inicjalizacyjny
        this.Text = "Moja Aplikacja";
        this.Size = new System.Drawing.Size(800, 600);
    }
}
```

23.2. Kontrolki

Kontrolki to elementy interfejsu użytkownika. Najważniejsze:

Label — wyświetla tekst:

```
// W Designer lub kodowo:
Label lblTytul = new Label();
lblTytul.Text = "Wprowadź dane:";
lblTytul.Location = new Point(10, 10);
lblTytul.Font = new Font("Arial", 12, FontStyle.Bold);
this.Controls.Add(lblTytul);
```

TextBox — pole tekstowe do wpisywania:

```

    TextBox txtImie = new TextBox();
    txtImie.Location = new Point(10, 40);
    txtImie.Width = 200;
    txtImie.PlaceholderText = "Wpisz imię...";
    this.Controls.Add(txtImie);

    // Odczyt wartości
    string imie = txtImie.Text;

    // Wieloliniowe
    TextBox txtOpis = new TextBox();
    txtOpis.Multiline = true;
    txtOpis.ScrollBars = ScrollBars.Vertical;
    txtOpis.Height = 100;

```

Button — przycisk:

```

    Button btnOk = new Button();
    btnOk.Text = "OK";
    btnOk.Location = new Point(10, 70);
    btnOk.Click += BtnOk_Click;    // przypisanie zdarzenia
    this.Controls.Add(btnOk);

    private void BtnOk_Click(object sender, EventArgs e)
    {
        MessageBox.Show($"Witaj, {txtImie.Text}!");
    }

```

ComboBox — lista rozwijana:

```

    ComboBox cboMiasto = new ComboBox();
    cboMiasto.Items.Add("Warszawa");
    cboMiasto.Items.Add("Kraków");
    cboMiasto.Items.AddRange(new object[] { "Gdańsk", "Wrocław", "Poznań" });
    cboMiasto.DropDownStyle = ComboBoxStyle.DropDownList;    // tylko wybór z listy

    string wybrane = cboMiasto.SelectedItem?.ToString();
    int indeks = cboMiasto.SelectedIndex;

```

ListBox — lista z wielokrotnym wyborem:

```

    ListBox lstImiona = new ListBox();
    lstImiona.Items.Add("Jan");
    lstImiona.Items.Add("Anna");
    lstImiona.SelectionMode = SelectionMode.MultiSimple;

    // Dodanie z kolekcji
    lstImiona.Items.AddRange(new object[] { "Piotr", "Maria" });

```

```
// Odczyt zaznaczonego
string zaznaczone = lstImiona.SelectedItem?.ToString();
```

24. Obsługa zdarzeń

Zdarzenia to reakcje aplikacji na akcje użytkownika.

24.1. Click

```
// Przypisanie w Designer – dwukrotne kliknięcie na przycisk
// Lub w kodzie:
btnDodaj.Click += BtnDodaj_Click;

private void BtnDodaj_Click(object sender, EventArgs e)
{
    string imie = txtImie.Text.Trim();
    if (string.IsNullOrEmpty(imie))
    {
        MessageBox.Show("Podaj imię!", "Błąd", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }
    lstImiona.Items.Add(imie);
    txtImie.Clear();
    txtImie.Focus();
}
```

24.2. TextChanged

Wywoływane przy każdej zmianie tekstu w TextBox:

```
txtSzukaj.TextChanged += TxtSzukaj_TextChanged;

private void TxtSzukaj_TextChanged(object sender, EventArgs e)
{
    string fraza = txtSzukaj.Text.ToLower();
    lstWyniki.Items.Clear();

    foreach (string imie in wszystkieImiona)
    {
        if (imie.ToLower().Contains(fraza))
            lstWyniki.Items.Add(imie);
    }
}
```

24.3. SelectedIndexChanged

Wywoływane przy zmianie zaznaczenia w ComboBox lub ListBox:

```
cboKlasa.SelectedIndexChanged += CboKlasa_SelectedIndexChanged;
```

```
private void CboKlasa_SelectedIndexChanged(object sender, EventArgs e)
{
    string wybranaKlasa = cboKlasa.SelectedItem?.ToString();
    if (wybranaKlasa == null) return;

    // Załaduj uczniów z wybranej klasy
    lblInfo.Text = $"Wybrano klasę: {wybranaKlasa}";
    ZaładujUczniow(wybranaKlasa);
}
```

25. DataGridView

DataGridView to kontrolka do wyświetlania danych w tabeli (siatkę).

25.1. Wyświetlanie danych

```
// W Designer dodaj kontrolkę DataGridView do formularza (np. dgvUczniowie)

// Wypełnianie z listy obiektów
private void ZaładujDane()
{
    List<Uczen> uczniowie = new List<Uczen>
    {
        new Uczen { Id = 1, Imie = "Jan", Nazwisko = "Kowalski", Srednia = 4.5 },
        new Uczen { Id = 2, Imie = "Anna", Nazwisko = "Nowak", Srednia = 5.0 }
    };

    dgvUczniowie.DataSource = uczniowie;

    // Formatowanie kolumn
    dgvUczniowie.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
    dgvUczniowie.ReadOnly = true;
    dgvUczniowie.AllowUserToAddRows = false;
    dgvUczniowie.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
    dgvUczniowie.Columns["Id"].HeaderText = "Numer";
    dgvUczniowie.Columns["Srednia"].DefaultCellStyle.Format = "F2";
}

// Odczyt zaznaczonego wiersza
private void BtnSzczegoly_Click(object sender, EventArgs e)
{
    if (dgvUczniowie.SelectedRows.Count == 0) return;

    DataGridViewRow wiersz = dgvUczniowie.SelectedRows[0];
    string imie = wiersz.Cells["Imie"].Value.ToString();
    MessageBox.Show($"Wybrałeś: {imie}");
}
```

25.2. Połączenie z bazą danych

```
private void ZaładujZBazy()
{
    string conn = "Server=localhost;Database=Szkola;Trusted_Connection=True;";

    using (SqlConnection connection = new SqlConnection(conn))
    {
        connection.Open();

        string sql = "SELECT Id, Imie, Nazwisko, Srednia FROM Uczniowie ORDER BY Nazwisko";
        using (SqlCommand cmd = new SqlCommand(sql, connection))
        {
            using (SqlDataReader reader = cmd.ExecuteReader())
            {
                // Czyścimy i budujemy tabelę ręcznie
                DataTable tabela = new DataTable();
                tabela.Load(reader);
                dgvUczniowie.DataSource = tabela;
            }
        }
    }

    // Konfiguracja po załadowaniu
    dgvUczniowie.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
    dgvUczniowie.ReadOnly = true;
    dgvUczniowie.AllowUserToAddRows = false;
}

// Alternatywa - SqlDataAdapter
private void ZaładujZBazySDA()
{
    string conn = "Server=localhost;Database=Szkola;Trusted_Connection=True;";
    string sql = "SELECT * FROM Uczniowie";

    using (SqlConnection connection = new SqlConnection(conn))
    {
        SqlDataAdapter adapter = new SqlDataAdapter(sql, connection);
        DataTable tabela = new DataTable();
        adapter.Fill(tabela);
        dgvUczniowie.DataSource = tabela;
    }
}
```